
Proyecto: Programación Orientada a Objetos en Python

Autor: David Burneo

Carrera / Curso: Computación “A”

Fecha: 16/06/2026

Introducción

Este proyecto documenta el uso de clases, herencia y validaciones en Python, aplicando conceptos de Programación Orientada a Objetos (POO). Se emplearon herramientas de visualización para comprender mejor la relación entre objetos, atributos y métodos.

Objetivos

- Implementar clases con atributos privados y propiedades.
 - Aplicar herencia entre clases (`Empleado` y `Docente`).
 - Validar datos mediante excepciones (`ValueError`).
 - Visualizar la ejecución y el estado de los objetos en memoria.
-

Desarrollo del Proyecto

1. Definición de la clase `Empleado`

The image shows a Python IDE with the following code:

```
Python 3.11
known limitations

21 @salario.setter
22 def salario(self, nuevo_salario):
23     if isinstance(nuevo_salario, (int, float)) and nuevo_salario > 0:
24         self.__salario = nuevo_salario
25     else:
26         raise ValueError("El salario debe ser un número positivo.")
27
28 def mostrar_info(self):
29     return f"Empleado: {self.nombre}, Salario: {self.salario}"
30
31 class Docente(Empleado):
32     def __init__(self, nombre, salario, materia):
33         super().__init__(nombre, salario)
34         self.materia = materia
35
36 @property
37 def materia(self):
38     return self.__materia
39
40 @materia.setter
```

Below the code, there is a visualization of the `Empleado` class and its instance. The **Frames** panel shows the `Global frame` and the `Empleado` instance. The **Objects** panel shows the `Empleado` class and its instance. The `Empleado` class is shown with the following attributes and methods:

Empleado class	
<code>__init__</code>	function <code>__init__(self, nombre, salario)</code>
<code>mostrar_info</code>	function <code>mostrar_info(self)</code>
<code>nombre</code>	property instance
<code>salario</code>	property instance

The `Empleado` instance is shown with the following attributes and methods:

Empleado instance	
<code>__init__</code>	function <code>__init__(self, nombre, salario)</code>
<code>mostrar_info</code>	function <code>mostrar_info(self)</code>
<code>nombre</code>	property instance
<code>salario</code>	property instance

At the bottom, there is a legend: a green arrow points to "line that just executed" and a red arrow points to "next line to execute".

Figura 1. Código de la clase `Empleado` y su representación en memoria.

2. Creación de la clase `Docente` (Herencia)

La clase `Docente` hereda de `Empleado` e incorpora el atributo `materia` con validaciones.

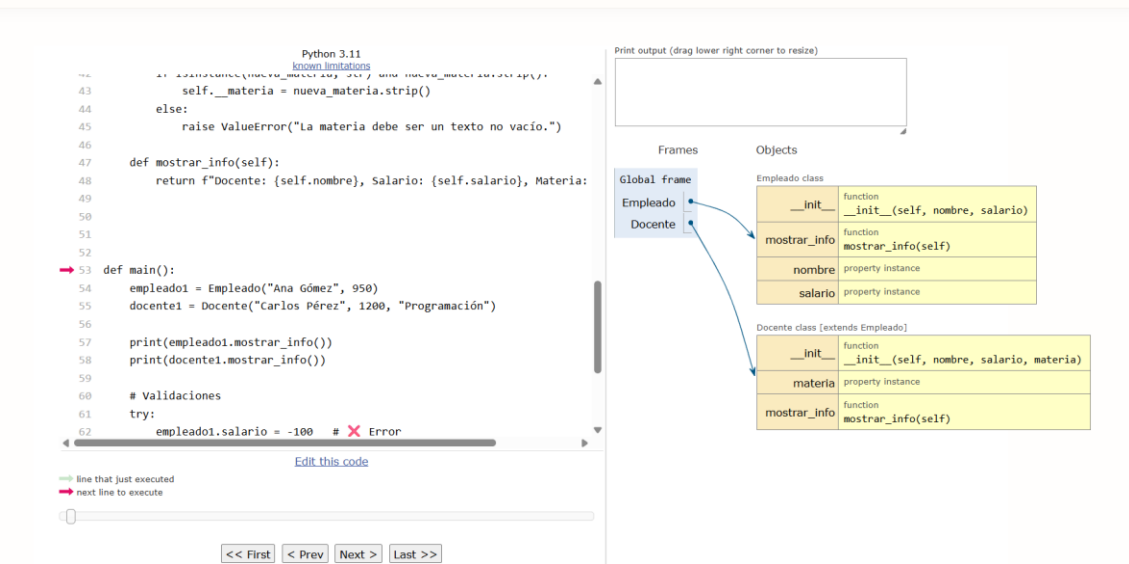


Figura 2. Código de la clase `Docente` y su representación en memoria.

3. Ejecución del programa y validaciones

Se instancian objetos y se prueban validaciones de salario y materia.

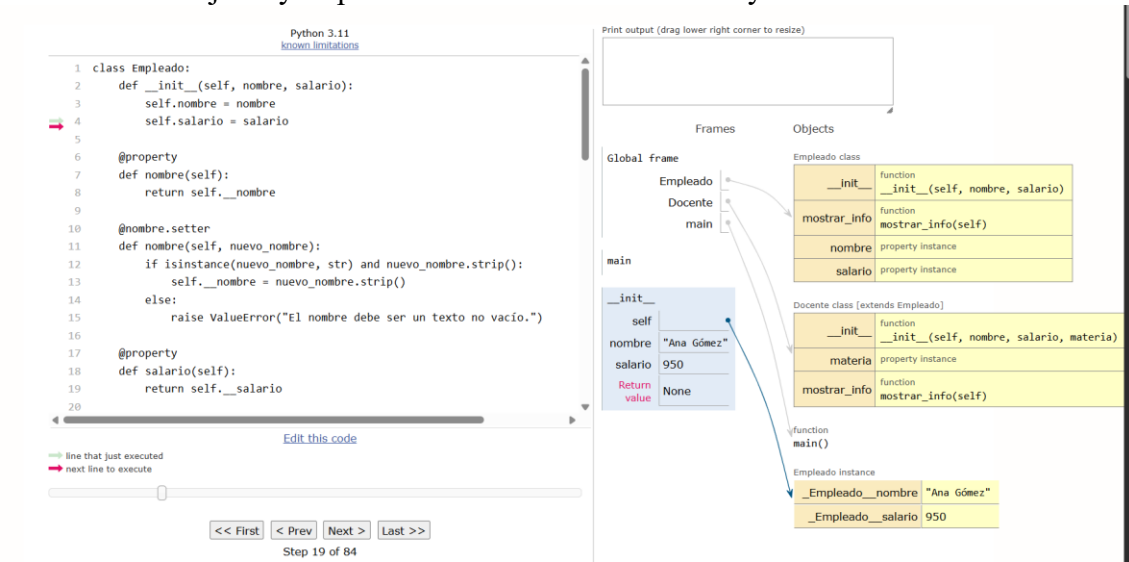


Figura 3. Ejecución del programa mostrando errores controlados.

4. Visualización de objetos y relaciones

Se utilizó un visualizador para observar cómo se almacenan los objetos y sus atributos en memoria.

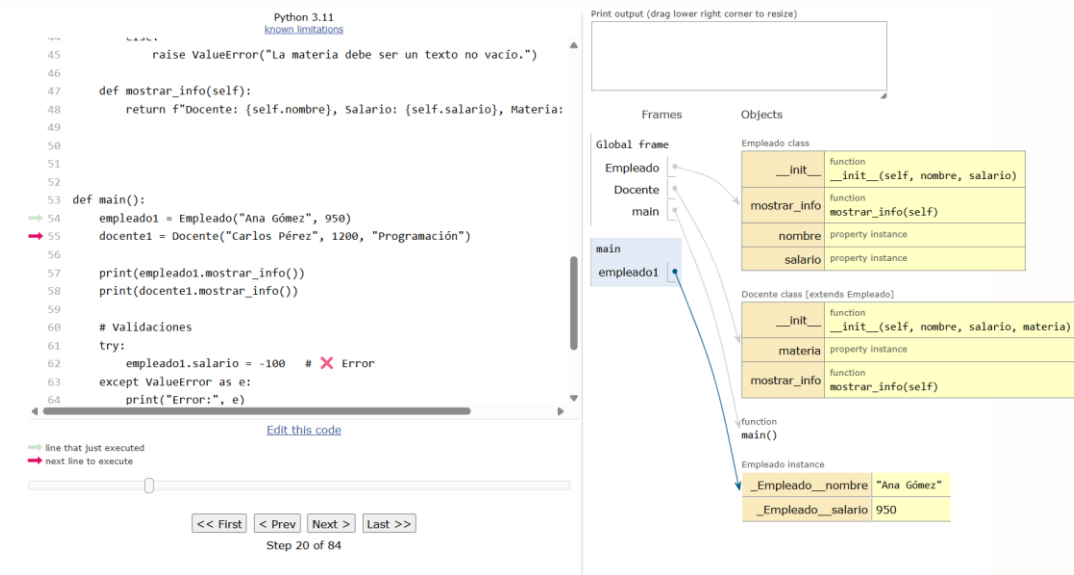
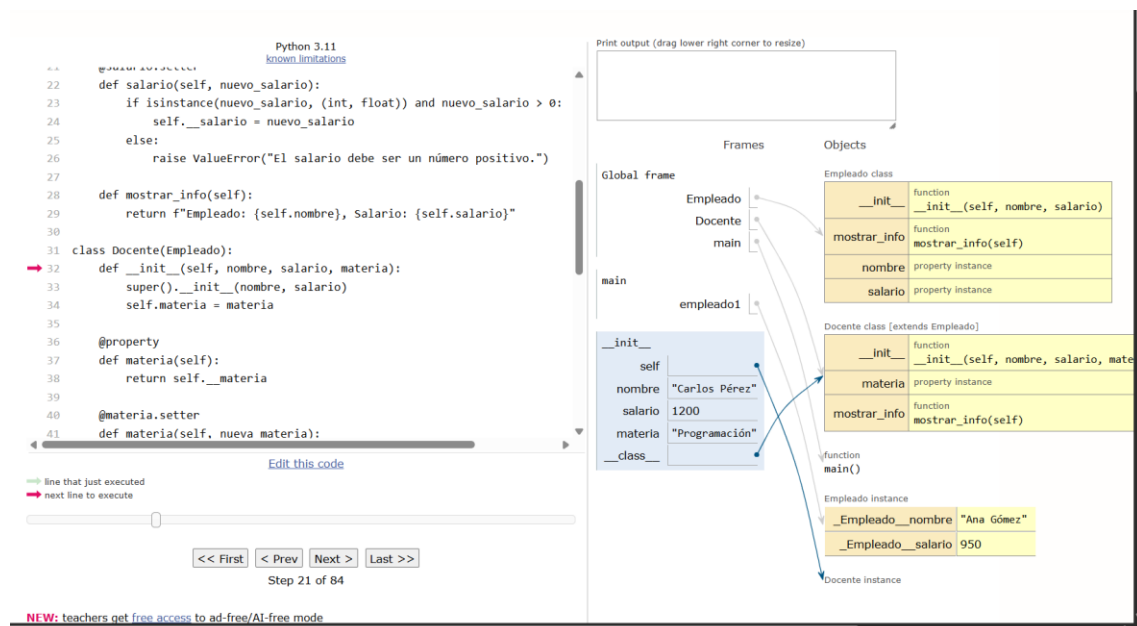


Figura 4. Diagrama de memoria con instancias de Empleado y Docente.



Conclusiones

- La herencia permite reutilizar código y extender funcionalidades.
 - Las propiedades (`@property`) facilitan el control de acceso y validación de atributos.
 - El uso de excepciones asegura la integridad de los datos.
 - Las herramientas de visualización son útiles para comprender la ejecución interna del programa.
-